

WEEK 1 (Day 3)

Loops in Python

Loops are used to execute a block of code repeatedly until a condition is satisfied. They help reduce repetition and make programs efficient.

1. for Loop

Used when the number of iterations is known.

Syntax:

for variable in sequence:

statements

```
for a in range(1,6):  
    print(a)
```

```
C:\Users\bhavn\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe  
1  
2  
3  
4  
5  
  
Process finished with exit code 0
```

```
x = int(input("Enter a number :"))  
for a in range(1,11):  
    print(x*a)
```

```
Enter a number : 8
```

```
8  
16  
24  
32  
40  
48  
56  
64  
72  
80
```

```
Process finished with exit code 0
```

2. while Loop

Used when the number of iterations is not fixed and depends on a condition.

Syntax:

while condition:

statements

```
count = 1  
while count < 6:  
    print(count)  
    count += 1
```

```
C:\Users\bhavn\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe
```

```
1  
2  
3  
4  
5
```

```
Process finished with exit code 0
```

```
n = int(input("Enter a number :"))
a = 1
while a <= 10:
    print(n,"x",a,"=",n*a )
    a += 1
```

```
Enter a number :6
```

```
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

```
Process finished with exit code 0
```

Nested Loops in Python

A **nested loop** means a **loop inside another loop**.

The inner loop executes completely for each iteration of the outer loop.

Syntax:

for outer_variable in sequence:

for inner_variable in sequence:

statements

OR

while condition:

while condition:

statements

```
for i in range (1,4):  
    for j in range (1,10):  
        print(j , end=" ")  
    print()
```

```
C:\Users\bhav\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe  
1 2 3 4 5 6 7 8 9  
1 2 3 4 5 6 7 8 9  
1 2 3 4 5 6 7 8 9  
  
Process finished with exit code 0
```

```
for i in range(1,6):  
    for j in range(1 , i+1):  
        print(j , end=" ")  
    print()
```

```
C:\Users\bhav\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe  
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5  
  
Process finished with exit code 0
```

For Loop with Conditional Statements in Python

A **for loop with conditional statements** means using conditions (if, if-else, if-elif-else) inside a for loop to control what happens during each iteration.

Syntax:

for variable in sequence:

if condition:

statements

```
for i in range(1, 101):  
    if i % 8 == 0 and i % 12 == 0:  
        print(i)
```

```
C:\Users\bhavn\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe  
24  
48  
72  
96  
  
Process finished with exit code 0
```

Break and continue in Python

Both break and continue are **loop control statements** used to change the normal flow of loops.

1. break Statement

The break statement is used to **stop the loop immediately** when a condition becomes true.

Syntax:

for variable in sequence:

if condition:

break

```
for i in range(1,10):  
    if i == 4:  
        print("This is my fav song")  
        break  
    else:  
        print(i)
```

```
C:\Users\bhavn\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe
1
2
3
This is my fav song

Process finished with exit code 0
```

2. continue Statement

The continue statement is used to **skip the current iteration** and move to the next iteration.

Syntax:

for variable in sequence:

 if condition:

 continue

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

```
C:\Users\bhavn\OneDrive\Desktop\DataAn1\.venv\Scripts\python.exe
1
2
4
5

Process finished with exit code 0
```